

A COST-EFFECTIVE, FAST, AND ROBUST ANNOTATION TOOL

Asaad F. Said, Vinay Kashyap, Namrata Choudhury, and Farshad Akhbari

Intel Corporation, Chandler, AZ 85226, USA

Asaad.f.said@intel.com

ABSTRACT

Deep learning requires huge datasets for training their models. In most cases, dataset generation is done manually or by using conventional approaches which are time consuming, costly, and prone to inaccuracy. There is an urgent need for developing fast, robust, and effective techniques to annotate each image with different labels and classes based on its contents. In this paper, we propose the use of in-house annotation tool that can be used for generating accurate datasets in a short time with minimal human interference. The proposed tool reads a captured video and the operator manually highlights the objects with their assigned classes in the initial frame. A robust and fast object tracking approach was developed to detect and track each highlighted object for subsequent frames. Experimental results using our annotation tool show 50x to 200x faster processing time compared to conventional annotation methods in addition to better performance with higher accuracy.

Index Terms— Object Tracking, Image Annotation, Template Matching, Kernelized Correlation Filter, Keypoints Matching, Multi-scale and Multi-class Object Tracking.

1. INTRODUCTION

A good annotation tool has a wide array of applications ranging from categorizing data for analysis in the field of data science, to generating labelled datasets for supervised learning in computer vision, especially in object recognition. The Deep Learning (DL) market has a huge potential across various industries such as advertisement, finance, and automotive. Due to the need to generate huge dataset for training DL models, some companies allocated significant money and resources for the image annotation task. In 2016, Mobileye had 600 people annotating images and they had plans to increase the number by 1,000 or more by 2017 [1]. Other companies use crowdsourcing platforms which provides the opportunity to outsource work to a large number of people and can help in generating labelled datasets. Although, in order to get reliable annotations, the same task needs to be allocated to multiple annotators [2]. This further increases the cost incurred for obtaining annotations. Annotated datasets are the primary requirement to generate ground truths for any supervised detection task, and it is not false to say that labeling is one of the most labor intensive tasks. Contemporary computer vision research has

demonstrated the importance of massive data sets of annotated images in results of ImageNet, PASCAL, LabelMe, SUN, etc. Most of these datasets consist of individual unrelated images and require some sort of intelligent recognition to annotate each frame, usually by a human. Ideally, the best annotations can be obtained by a trained person annotating all the datasets, but this is an incredibly tedious task. Including multiple resources using crowdsourced platforms for annotating datasets, increases the cost, and usually requires immense efforts to ensure quality control measures.

Dynamic environments in video data make frame by frame labeling necessary but inefficient for manual labor, as there are a lot of redundant objects. LabelMe [3] was presented as an open platform for dense polygon labeling on static images using homography preserving linear interpolation. ViPER [4], a flexible video annotation system optimized manual spatial labeling for every frame. FlowBoost [5] is a tool that can annotate videos from sparse set of key frame annotations. These techniques have some sort of linear interpolation or a very basic tracker which pursue the selected objects for a few frames, but still require considerable interference from a user. Vondrick et al [6] introduced the VATIC annotation framework based on linear tracking of discriminative object templates and also their research of annotation based on crowdsourced marketplaces. They conclude that video annotation requires specialized skill and most workers on crowdsourced platforms are poor annotators, mandating robust quality control protocols. The authors also mentioned that automated tracking for 102 hours of video on Amazon MTurk platform, with users being paid \$0.02 for every HIT, cost them about \$500. They also added that the users complained that the compensation rate was not sustainable and demanded at least \$0.10 per HIT which would have cost about \$2500 for that dataset alone. Unsurprisingly, the quality of the annotations also improved when the workers were compensated more. This outlines the cost and quality control aspects required for labeling massive amounts of data. VATIC annotator assumes annotated object moves in a linear motion with constant speed. If one of this fails, it adds more annotation nodes and hence more time.

Another major factor is that, videos obtained from automotive platforms are moving with respect to the environment, with constant changes in background, addition/deletion of multiple objects and constant scale changes, which severely restrict tracking. These datasets also



Fig. 1: User Interface of our in-house Annotation Tool

tend to be extremely large and unorganized, making the quality control requirements even stricter. In this paper we present a cost-effective and highly optimized semi-automated video annotation tool, utilizing robust object tracking.

The paper is arranged as follows: section 2 gives an overview of the proposed annotation tool. Section 3 provides an overview of the implemented visual object tracking approaches. Results are presented in section 4. Summary is presented in section 5.

2. IMAGE ANNOTATION TOOL

After studying different annotation tools, we came up to a conclusion that the existing tools are not automated, time consuming, costly, and exhibit many weakness points. Annotating each object in video frames is a tedious job to do manually. It can lead to human errors since operator working on a longer video gets tired over time and may lead to inaccurate annotation. Our main goal in this work is to build a robust, fast, and cost-effective annotation tool that automates the labeling process with very minimal interference of the operator. The main goal is to have an interface that allows the operator to annotate any number of object(s) within the same class or different classes as an initialization step. This enables more flexibility to select from the listed classes or add a new class to the list. The proposed User Interface (UI) is shown in Fig.1. Operator is allowed to select as many classes as desired to the list. Each class is highlighted with a unique color. Due to the nature of temporal information in each video, one can benefit from the gradual changes between consecutive frames to develop an approach that detects and highlights objects in the new frames, based on the detected objects from previous frames. The proposed scheme uses the manually highlighted objects from the first

frame as initialization masks, then track them over time using a robust object tracking approach. Further details about the tracking approaches are provided in section 3. Tracking can be paused and then resumed at any point based on the operator preferences (add new object/class to the list). At each frame, the tracking results are refreshed and displayed automatically on the UI. During the processing, the UI saves the result of each frame in an annotation file that can be loaded or accessed at any time. The user has the flexibility to navigate between different frames once the saved annotation file is loaded in the UI. The tool also provides the extracted annotations in CSV format (containing the class name and coordinates of the annotated object), so that the annotations can be used in external applications, by parsing the CSV file appropriately. It even provides cropped images of the objects, which is the desired format for training object recognition pipelines in certain deep learning frameworks, like Caffe. This is a huge advantage when compared to current annotation tools in the market, which only provide annotation data in specific formats, like the Visual Bounding Box in the Caltech Annotator.

3. VISUAL OBJECT TRACKING

Object detection and tracking is one of fundamental tasks implemented in this paper to help automate the annotation process. By highlighting the bounding boxes of different objects in the first frame, one can use these boxes as an initialization step for any Visual Object Tracking (VOT) procedure in order to detect and track each objects separately over time. It is very important to have a VOT procedure that detects and tracks objects accurately for a longer period of time in order to significantly minimize the operator work. Several VOT procedures were implemented and compared to finalize on a robust and fast approach that provides

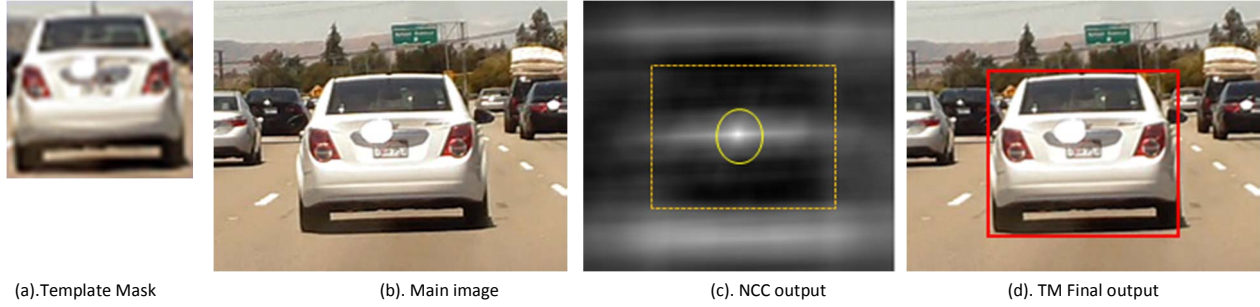


Fig. 2: Steps of object tracking using Template Matching

long-term tracking with an accurate detection of different objects/classes.

3.1. Template Matching

Template Matching (TM) is very common and heavily used approach in image processing and computer vision applications. The idea of the TM is to check the similarities between two different images: the first represents the main image and the second represents the template image which has a smaller size compared to the main image. Similarity between two images can be measured by different approaches. Normalized cross-correlation (NCC) is the most commonly used approach to measure the similarity in TM.

$$NCC = \frac{\sum_i (X_i - \mu_X)(T_i - \mu_T)}{\sum_i (X_i - \mu_X)^2 \sum_i (T_i - \mu_T)^2} \quad (1)$$

Where X and T represent the main and template images, respectively. μ_X and μ_T represent the mean values for X and T images, respectively. Fig. 2 shows the steps of using TM for object tracking. The NCC output between the template in Fig. 2(a) and the main image in Fig. 2(b) is shown in Fig. 2(c). The brightest point in the NCC output represents the highest correlation point which can be used to extract the object's center point and therefore the bounding box as shown in Fig. 2(d). The newly detected object is automatically extracted to be used as a new template mask for the coming frame. The template mask is updated at each frame to compensate the pose variation from frame to frame. At each iteration, the resulted NCC value has to be checked to make sure it is higher than the predefined value (95%), to ensure a good matching, otherwise the current frame is presented to the operator to highlight the new template. The scaling issue can be mitigated by using multi-scale template matching in which different sizes of the template are used and the one that produces the highest NCC value among the used sizes is selected to be the right size. Fig. 3 shows the VOT result using multi-scale template matching. The image on left represents the first frame with the initialization for the template mask. The middle image shows the intermediate VOT results, while the image on the right shows the last frame where the object still detected for 1200 frames using the first initialization, without human interference in between. The multi-scale template matching approach shows better localization of the targeted object. However, there are some challenges, such as, pose variation, rotation, and drifting. Drifting is one of the main issues in TM due to the use of the template from previous frame. To minimize the drifting, a set of templates with different poses can be used, however this adds more computations. Rotated template

with different angles can be used to minimize the issues related to the rotation. To resolve the aforementioned issues in TM, it requires several set of templates with significant computations.

3.2. Adaptive Histogram based Matching

The image histogram doesn't show significant changes when the image is rotated, which makes it a good candidate for rotation invariance. This is an important feature that can be used to propose a new matching approach based on the histogram to resolve the rotation issues between the template and the main image. The procedure uses the template mask to calculate the normalized histogram [7], then stores it in a buffer for further processing. To extract the location of the object in the new frame, a sliding window approach is used to scan over the frame to calculate the normalized histogram for each window. The resulting normalized histograms are compared against the ones stored in the buffer using distance functions. Different distance functions were used, however we adopted 1D NCC to measure the similarities between the histograms. The window that resulted in the highest NCC is selected to be the candidate object in the output. Due to the small scale variation between consecutive frames, the scaling issue can be resolved by proposing the use of 3 different sizes of sliding windows which are 0.95, 1, and 1.05 w.r.t. the template size from previous frame. To resolve the issues related to pose variation, we added the learning stage in the proposed approach. In the learning stage, if the histogram of the detected object shows strong correlation with the ones in the buffer, the new histogram is added to the buffer. If the buffer is full, the one with the weak correlation is replaced by the new one with strong correlation. This helps to enable the addition of new histogram to the buffer where a weight is assigned to each histogram based on the similarity. To minimize the drifting issue, higher weight was assigned to the original template's histogram in the buffer. Another procedure to minimize drifting is the use of Kalman filter. Fig. 4 shows the VOT result using histogram-based matching. The method shows accurate detection with long-term tracking that lasted for 2280 frames using one time initialization. The main drawbacks of the methods are computation time in addition to its sensitivity to shadow or objects with similar gray level within the ROI area.

3.3. Keypoint Feature Matching

Most traditional object trackers struggle while tracking objects with scale variations. Hence, we tested keypoint feature based tracker which estimated the affine transformation between



Fig. 3: Object tracking using Multi-scale Template Matching



Fig. 4: Object tracking using Adaptive Histogram-based Matching.

matched features in successive frames. Once the template mask has been selected in the first frame, features were detected and extracted using Scale-invariant Feature Transform (SIFT). In subsequent frames, SIFT features for the complete image or ROI are extracted and feature points matching is performed between the template and ROI. SIFT detector and descriptor is scale and rotation invariant. The matching feature points are used to get the affine transformation parameters (scale, rotation, and shift) and therefore applied on the ROI to extract the object. This procedure is done for each object in the first frame and the features are propagated till there are not enough matching points between consecutive frames. We estimate that affine transformation needs at least 4 good feature points to be matched for the parameters estimation. We observed that tracking performed well if there were good features extracted from the selected bounding box. Based on our testing on some of the captured videos, keypoints matching is sensitive to the size of the object in ROI where smaller objects don't have enough details and produces less matching points. This is one of the main drawbacks of this approach. Also any mismatch of features due to repeated patterns etc., causes the estimated transformation to degenerate and results in a faulty detection. Since feature matching is not invariant to deformable models, tracking pedestrians is not very accurate due to constant motion of different parts of the body. Fig. 5 shows the performance results using keypoints matching where detection and tracking is good as long as the object contains more details. If the object is smaller or doesn't have more details, the detection looks faulty, as shown on the right image in Fig. 5. This makes this approach not a good candidate for long-term tracking.

3.4. Kernelized Correlation Filter

Recently, VOT using Kernelized Correlation Filter (KCF) has attracted the attention of many researchers due to its simplicity, speed, efficiency, and robustness in object tracking [8-9]. VOT using KCF consists of 3 different stages. The 1st stage applies resizing and feature extraction to the highlighted templates. Each template is represented by a feature vector using HOG. The 2nd stage applies correlation between the feature vectors of the

template and the test images. The matching point is selected based on the highest output response of the correlation. The 3rd stage is used for adaptive learning where the feature vector is updated if the considered template shows strong matching. The adaptive learning stage helps significantly to resolve issues related to drift and pose variation over time. To resolve the scaling variation issue, 3 different scales (0.95, 1, 1.05) w.r.t to the previously detected template are used in the scheme. The right scale is selected based on the highest correlation response.

The VOT using KCF was tested using several videos and showed very promising results, compared to the aforementioned approaches. Further modifications were included in the KCF approach to enable multi-objects and multi-class tracking. Several objects within the same class or different classes can be easily tracked without any issues. It enables tracking of overlapped objects from different classes. The correlation response is used as a stopping criteria to detect the occlusion or out-of-view scenarios in each frame. The stopping criteria is based on the confidence score between detected objects and the trained templates collected during the tracking and learning process. The learning stage was further modified to only include objects with high confidence or correlation response. The used modifications help in eliminating any distorted objects from biasing the learning stage. Fig. 6 shows the performance of the KCF in tracking different objects accurately even with the presence of scale and pose variations. A single object was detected and tracked for the entire video of 6400 frames using a single initialization. This makes the modified KCF an excellent long-term tracking approach and a good candidate for our annotation tool.

The modified KCF tracker was further enhanced for deformable objects such as pedestrians. Due to constant change in the overall shape of these objects as they move, a new approach was implemented to identify the non-changing part of pedestrians for more accurate tracking. This approach is based on Da Vinci's Vitruvian man model shown in Fig. 7. It divides human body into its individual parts (ROIs), as depicted in Figure 8. In this approach, only the head and upper body of the pedestrian is used for detection and tracking, eliminating the need to adapt to the



Fig. 5: Visual Object Tracking using Keypoints Matching.



Fig. 6: Visual Object Tracking Scheme using Kernelized Correlation Filter.

deformable parts of the object, i.e. hands and legs. Each ROI is individually tracked by the enhanced KCF while maintaining the relationship with the original mask. The algorithm constantly checks each ROI with its own template over time to maintain correlation. The final bounding box around the pedestrian is automatically adjusted and updated based on the tracking outcome. Fig. 9 shows the performance of the modified tracker in pedestrian tracking.

4. SIMULATION RESULTS

The modified KCF was integrated in our annotation tool. The tool is being used for testing different objects annotations including cars, pedestrians, traffic lights, traffic signs, etc. All rigid objects are using the modified KCF tracker except for pedestrian which use Vitruvian man model integrated with the modified KCF for better tracking, due to deformation and pose variation in pedestrian. Fig. 1 shows the performance of the annotation tool in accurately detecting and tracking different objects and classes within the frame (cars, pedestrians, traffic signs, etc.). Members of our team generated datasets using manual annotations which are needed for many projects. Few engineers and interns spent significant time annotating different videos. The time spent in the annotation process varied between them due to their familiarities with the task. Considering a video with an average of 3 objects/frame, there were variations in the time spent in the annotation process, ranging from 80 frames/h for a beginner to 120 frames/h for an advanced operator. With an average of 100 frames/h for annotation, it will take 100h to annotate a single video with 10k frames. It will take 20 days for a trained person working 5h a day to finish the annotation. In some cases to ensure the accuracy, more than one operator needs to annotate the same video which increases the time of the annotation process. This time can be saved by using our annotation tool. The VOT approach takes 11ms/frame on average on a Haswell laptop (2.5GHZ Intel core i5 with 4G

Table 1: Performance of the annotation tool vs. manual annotation.

Videos	Available Classes				# Frames	Manual Ann. Time (min)	Annotation Tool	
	Car	Ped.	TL	TS			Time (min)	Speed (x)
HW	Yes	No	No	Yes	18,000	15,000 (250h)	105 (1.75h)	142.9
Urban	Yes	Yes	No	Yes	1,330	1,108 (18.5h)	25 (0.4h)	44.3
HW	Yes	No	Yes	Yes	7,311	6,093 (101.5h)	62 (1h)	98.3
Overall					26,641	22,201 (370h)	192 (3.2h)	115

RAM). Adding the time taken for display and initialization cases will end up roughly between 30min to 2h based on how much time the operator has to interfere for initialization. This results in 50x to 200x reduction in time and therefore huge reduction in cost. Further analysis illustrating the performance of the annotation tool is provided in Table 1, where the proposed annotation tool was tested using several video samples with different classes (cars, pedestrians, traffic signs and traffic lights). The tool annotated 26,641 frames in 192min, with minimal human intervention. It is capable of annotating 8,325 frames/h with an average speed of 115 times faster than the time spent by manual annotation.

5. CONCLUSION

In this paper, we introduced an in-house solution for image annotation. The tool provides an excellent solution to generate an accurate dataset in a short time with minimal human intervention. It is a cost-effective, highly optimized, and accurate solution. The tool is currently used by our team for generating different datasets to be used in different projects. Future improvements are planned to be included in the tool to enable an automatic initialization which makes it fully automated. The initialization process can be replaced by one of the advanced multi-object detection approaches and can be scheduled to run in cases where new objects are introduced to the scene.

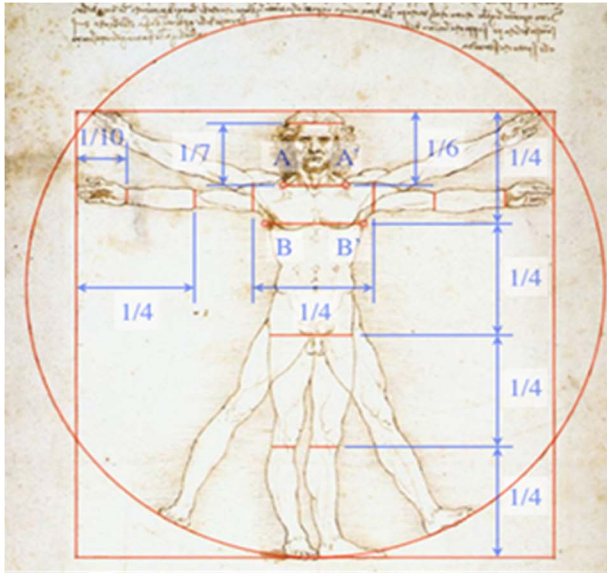


Fig. 7 Vitruvian Man model body part ratio by Da Vinci

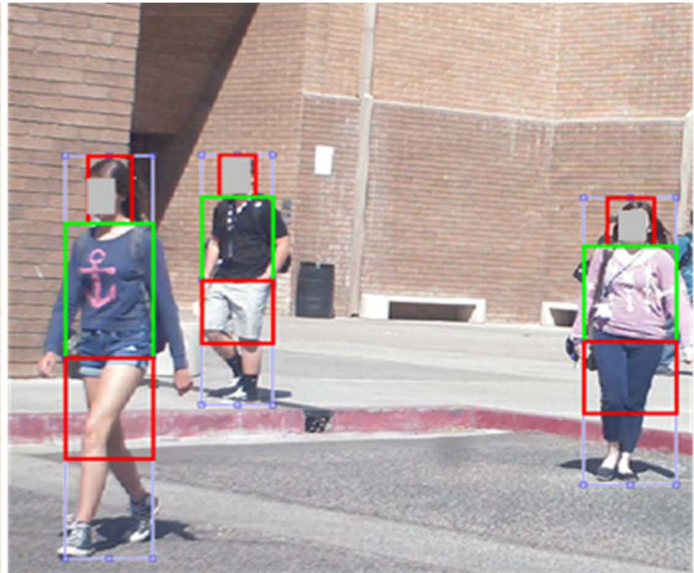


Fig. 8 ROIs extraction using Vitruvian Man model.

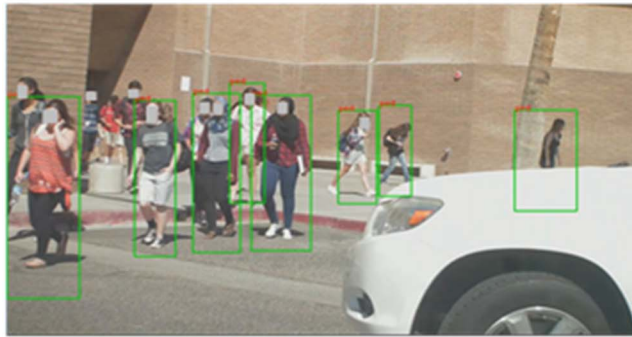


Fig. 9 Pedestrian Tracking using the proposed method using Vitruvian man model

6. REFERENCES

- [1] Prof. Amnon Shashua's talk, Mobileye's Co-founder, <https://www.youtube.com/watch?v=GCMXXXmxG-I>
- [2] Welinder, P. Perona, P. "Online crowdsourcing: rating annotators and obtaining cost-effective labels", CVPRW, 2010
- [3] Torralba A, Russell B, Yuen J "Labelme: online image annotation and applications", 2010.
- [4] Mihalcik D, Doermann D "The Design and Implementation of ViPER", 2010.
- [5] Ali K, Hasler D, Fleuret F "Flowboost- appearance learning from sparsely annotated video". IEEE CVPR, 2011.
- [6] Vondrick C, Ramanan D, Patterson D "Efficiently Scaling up Crowdsourced Video Annotation", IJCV, 2012.
- [7] Rafael C. Gonzalez and R. E. Woods, Digital Image Processing: Prentice Hall, 2003.
- [8] J. F. Henriques, R. Caseiro, P. Martins, and J. Batista. "High-speed tracking with kernelized correlation filters". *TPAMI*, 2015.
- [9] Guibo Zhu, Jinqiao Wang, Yi Wu and Hanqing Lu, "Collaborative Correlation Tracking", BMVC 2015.